

HW14

Tguo67

Question(s):

Write a paper review for [Prism: Unleashing GPU Sharing for Cost-Efficient Multi-LLM Serving](#)[Links to an external site.](#), following the format below:

Overall score (1: reject, 2: weak reject, 3: major revision, 4: minor revision, 5: accept)

Summary of the paper (be concise, in one paragraph)

Reasons for accepting the paper (list in a pointed format, i.e., "- reason1, - reason 2")

Reasons for rejecting the paper (the same as the reasons for acceptance)

Comments for the authors (the detailed opinions on this paper, including explaining your score, asking questions, suggesting improvements, etc.)

Answers:

1. Overall score

Minor revision

2. Summary of the paper

This paper presents Prism, a system for cost-efficient multi-LLM serving that enables flexible GPU sharing across many models. Prism introduces a virtual-memory-based KV cache layer so that KV memory can be dynamically allocated and reclaimed across models, plus an engine pool and fast multi-GPU weight loading to reduce model activation latency. A two-level scheduler (cluster-level model placement and GPU-local SLO-aware scheduling) coordinates memory and compute to improve SLO attainment. Experiments on multi-model traces show substantially higher GPU utilization, better SLO satisfaction, and lower cost compared to existing multi-LLM serving baselines.

3. Reasons for accepting the paper

- Tackles a timely and important problem: cost-efficient serving of many LLMs with long-tail popularity.
- Presents a clear systems contribution: a KV cache shim that unleashes cross-model GPU memory sharing with minimal engine changes.

4. Reasons for rejecting the paper

- Evaluation is done on a single backend; generality to other engines is argued but not demonstrated.
- Assumes relatively powerful hardware (e.g., fast interconnect) and does not fully explore behavior on weaker setups.

5. Comments for the authors

Prism is a well-motivated and solidly engineered system, and the main idea of VMM-based KV cache sharing plus SLO-aware scheduling is convincing. Maybe it is good to add either a second serving backend or a more detailed analysis of what is needed to integrate with other engines